SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	C.Mokshitha reddy	**Reg. No.:**	192571054
Section / Batch:		**Date:**	04/09/2026

Code of Conduct

I C.Mokshitha reddy (192571054) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Mokshu

Instructions

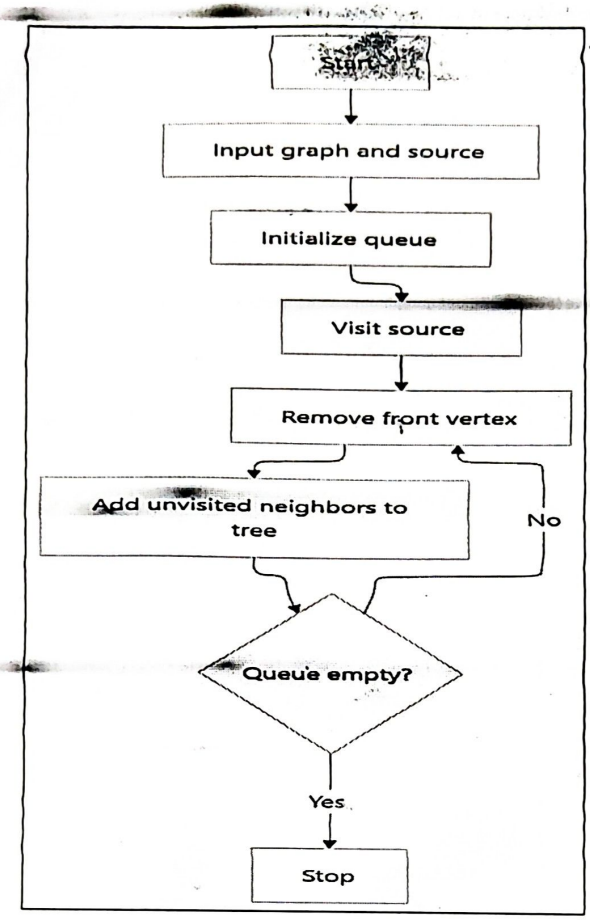
- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Pseudocode Writing Task

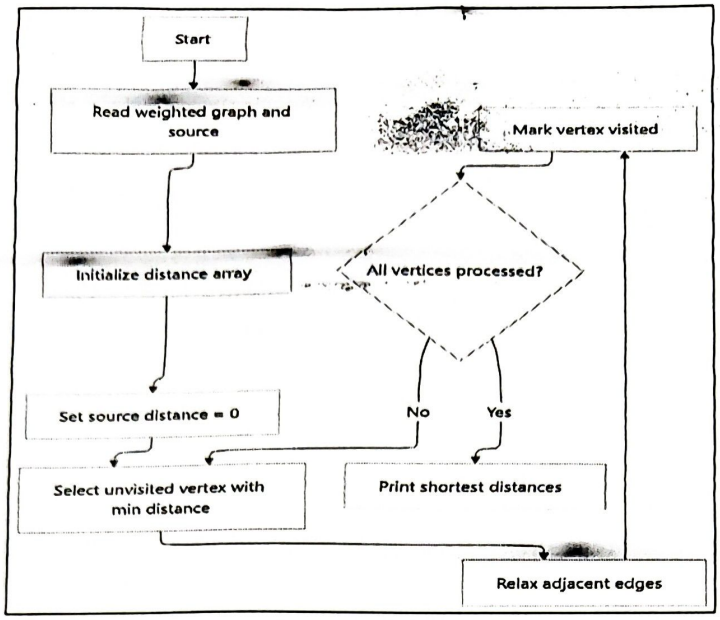
1. Convert the flowchart for generating a BFS tree into code. (20 Marks)

Flowchart Diagram:



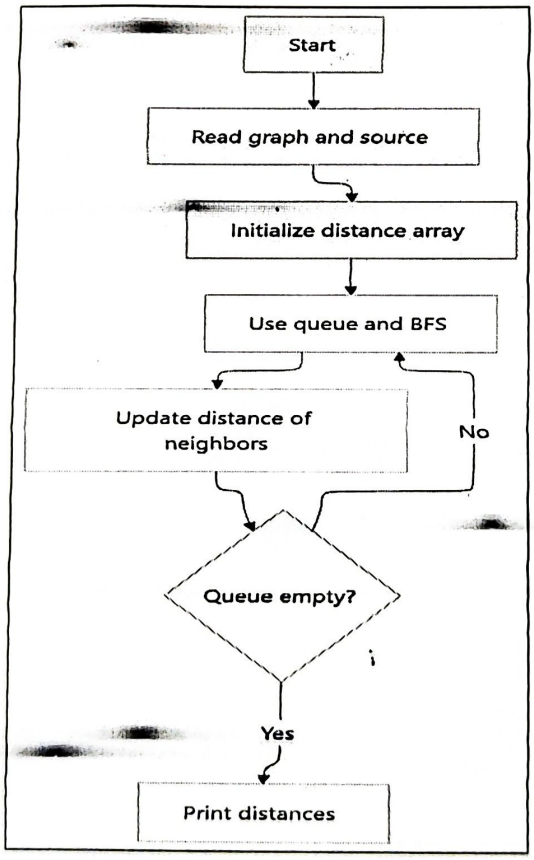
2. Convert the flowchart for Dijkstra's shortest path algorithm into code. (20 Marks)

Flowchart Diagram:



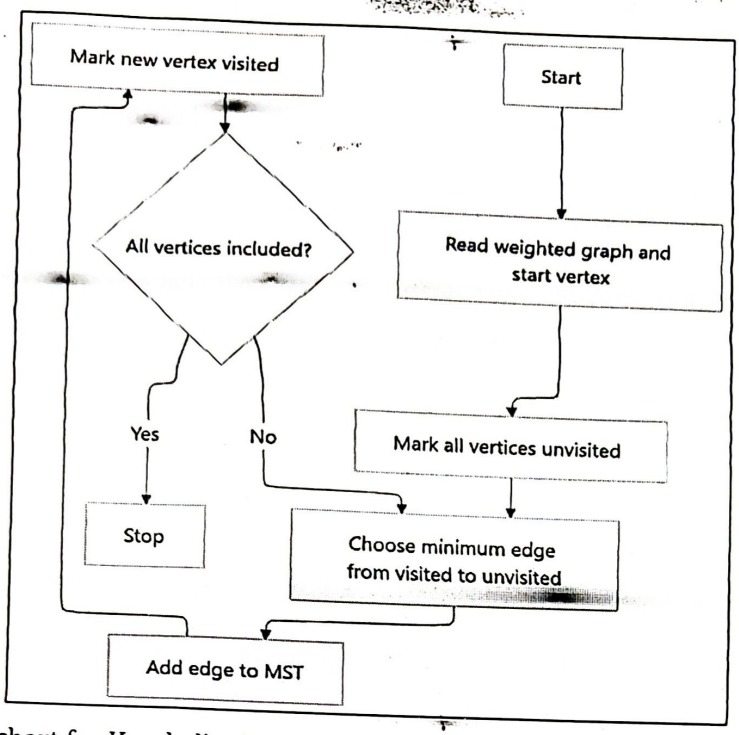
3. Convert the flowchart for shortest path in an unweighted graph into code. (20 Marks)

Flowchart Diagram:



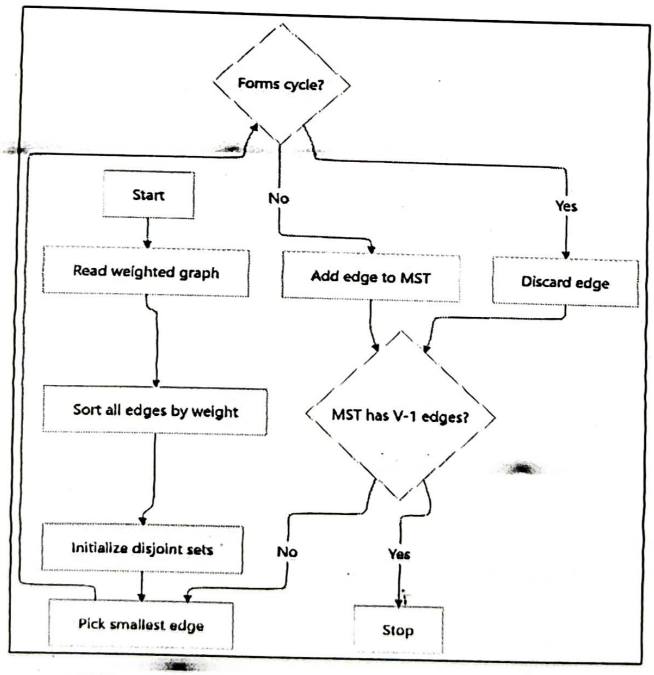
4. Convert the flowchart for Prim's algorithm into code. (20 Marks)

Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

BFS - code

```
#include<stdio.h>
int main() {
    int n, a[10][10], v[10] = {0};
    int q[10], f=0, r=0, s, i;
    Scanf("%d", &n);
    for(i=0, i<n, i++)
        for(int j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    scanf("%d", &s);
    q[r++] = s;
    v[s] = 1;
    Printf("BFS: ");
    while (f<r) {
        s = q[f++];
        Printf("BFS:")
        while (f<r) {
            s = q[f++];
            Printf("%d", s);
            for (i=0; i<n; i++){
                if (a[s][i]&&!v[i]) {
                    q[r++] = i;
                    v[i] = 1;
                }
            }
        }
    }
```

```
    return 0;
}
```

Input:-

```
5
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0
0
```

Output:-

BFS: 0 1 2 3 4

2. Dijkstra-code

```
#include<stdio.h>
int main()
    int n, a[10][10], d[10], v[10]={0};
    int s, i, j, min, u;
    scanf("%d", &n);
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    scanf("%d", &s);
    for(i=0; i<n; i++)
        d[i]=999;
    d[s]=0;
```

```
for (i = 0; i<n; i++) {
    min = 999;
    u = -1;
    for (j = 0; j<n; j++)
        if (!v[j] && d[j] < min)
            min = d[j], u = j;
    v[u] = 1;
    for (j = 0; j<n; j++)
        if (a[u][j] && d[u] + a[u][j] < d[j])
            d[j] = d[u] + a[u][j];
}
    for (i = 0; i<n; i++)
        Printf ("%d -> %d = %d\n", s, i, d[i]);
    return 0;
}
```

Input:-

5
0 10 3 0 0
10 0 1 2 0
3 1 0 8 2
0 2 8 0 7
0 0 2 7 0
0

Output:-

0 -> 0 = 0

0 -> 1 = 4

0 -> 2 = 3

0 -> 3 = 6

0 -> 4 = 5

3. unweighed shortest path

```
# include <stdio.h>
# define MAX 10
  int main() {
      int n, graph[MAX][MAX], dist[MAX], q[MAX];
      int visited[MAX] = {0}, front=0, rear=0, src;
      scanf("%d", &n);
      for(int i=0; i<n; i++)
          for(int j=0; j<n; j++)
          scanf("%d", &graph[i][j]);
      scanf("%d", &src);
      for(int i=0; i<n; i++) dist[i] = -1;
       q[rear++] = src;
       visited[src] = 1;
       dist[src] = 0;
       while(front<rear) {
           int u=q[front++];
           for(int v=0; v<n; v++) {
               if(graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
            dist[v] = dist[u] + 1;
            q[rear++] = v;
        }
    }
}

printf("shortest distances: \n");
for(int i=0; i<n; i++)
    printf("%d -> %d = %d\n", src, i, dist[i]);
return 0;
}
```

Input:-

```
5
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
0
```

Output:-

Shortest distances:

0 -> 0 = 0
0 -> 1 = 1
0 -> 2 = 1
0 -> 3 = 2
0 -> 4 = 2

4. Prim's Algorithm.

```
#include<stdio.h>
#define INF 9999
#define MAX 10
int main(){
    int n, cost[MAX][MAX], selected[MAX]={0};
    int edges=0, total=0;
    Scanf("%d",&n);
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            Scanf("%d",&cost[i][j]);
    Selected[0]=1;
    Printf("edges in MST :\n");
    while(edges-<n){
        int min=INF, x=-1, y=-1;
        for(int i=0; i<n; i++).
            if(selected[i])
                for(int j=0; j<n; j++)
                    if(!selected[j] && cost[i][j]<min){
                        min=cost[i][j];
                        x=i;
                        y=j;
                    }
    Printf("%d - %d : %d\n", x, y, min);
```

```
total += min;
selected[y] = 1;
edges++;
}
printf("Total MST cost = %d\n", total);
return 0;
}
```

Input:-

```
5
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
```

output:-

Edges in MST:

0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6

Total MST cost = 16

5. **kruskal's Algorithm:-**

```
#include<stdio.h>
struct Edge{
    int u, v, w;
```

```
};

int parent[20];

int find(int x){
    while(parent[x]!=x)
        x = parent[x];
    return x;
}

Void unite(int a, int b){
    Parent[find(a)] = find(b);
}

Void unite(int a, int b){
    Parent[find(a)] = find(b);
}

int main(){
    int n, e, count = 0, total = 0
    Struct edge a[50], temp;
    Scanf("%d, %d", &n, &e);
    for(int i=0; i<e; i++)
        Scanf("%d %d %d", &a[i].u, &a[i].v, &a[i].w);
    for(int i=0; i<n; i++)
        Parent[i] = i;
    for(int i=0; i<e-1; i++)
        for(int j=i+1; j<e; j++)
```

```
if (a[i].w > a[j].w){
    temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}

printf ("MST: \n");
for (int i=0; i<e && count<n-1; i++) {
    if (find(a[i].u) != find (a[i].v)) {
        Printf ("%d - %d : %d \n", a[i].u, a[i].v, a[i].w);
        total += a[i].w;
        unite(a[i].u, a[i].v);
        count++;
    }
}
printf ("total cost = %d", total);
return 0;
}
```

Input :-

5 7

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

Output :-

MST :

0 - 1 : 2

1 - 2 : 3

1 - 4 : 5

0 - 3 : 6

Total cost = 16